UDESC



Rust

Cristiano Damiani Vasconcellos
cristiano.vasconcellos@udesc.br

UDESC

Tipos Primitivos

Type	size_of::<Type>()
()	0
bool	1
u8	1
u16	2
u32	4
u64	8
u128	16
i8	1
i16	2
i32	4
i64	8
i128	16
f32	4
f64	8
char	4
usize, isize	Palavra do proc.

```
fn main() {
    let x:u32 = 10;
    println!("{} - {}", size_of::<u32>(),
                        size_of_val(&x));

}
```

UDESC

Declaração de variáveis

Variáveis mutáveis devem ser declaradas explicitamente:

```
fn main() {
    let x = 10;
    let y:u32 = 20;
    let mut z = 30;
    z = z + 1;
    println!("{x} - {y} – {z}");
}
```

UDESC

Referências

Sendo T um tipo, referências podem ser declaradas de duas formas:

&T – referência imutável.

&mut T – referência mutável (exclusiva).

Apenas um pode atualizar ou vários podem ler.

```
fn main() {
  let x:u64 = 10;
  let mut y:u64 = 20;
  let p1: &u64 = &x;
  let p2: &u64 = &x;
  let p3: &mut u64 = &mut y;
  *p3  = *p3 + 1;
  println!("{p1} - {p2} -{p3} ");
 }
```

UDESC

&str – String slice

Uma string slice é uma referência é uma referência uma sequência de caracteres codificada em UTF-8.

```
fn main() {
  let s1 = "Computacao";
  let s2 = &s1[6..];
  println!("{s1} - {} - {}", size_of_val(s1), size_of_val(&s1));
  println!("{s2} - {} - {}", size_of_val(s2), size_of_val(&s2));
 }
```

UDESC

UTF-8

É uma codificação de comprimento variável para caracteres codificados em *Unicode em que os* caracteres que fazem parte da tabela ASCII (7-bits) são codificados da mesma forma.

Byte 1	Byte 2	Byte 3	Byte 4
0xxxxxxx			
110xxxxx	10xxxxxx		
1110xxxx	10xxxxxx	10xxxxxx	
11110xxx	10xxxxxx	10xxxxxx	10xxxxxx

UDESC

Literais

Literal	Type
1_234	i32
1.5	f64
1.2-e10	f64
0b1000_0000	i32
0xFF	i32
0o17	i32
b’a’	u8
1.2f32	f32
16u32	u32
1u64	u64
‘a’	char
true	bool
“teste”	&str
[1,2,3]	[i32; 3]

```
fn main() {
    let x = 10i64;
    println!("{x}");

}
```

UDESC

Expressões

Em Rust boa parte das construções sintáticas são expressões:

```
fn main() {
    let a = 10;
    let b = if a < 10 {1} else {2}; // Expressão if
    println!("{b}");
}
```

```
fn main() {
    let a = 10;
    let b = if a < 10 {1} // Erro
    println!("{b}");
}
```

```
fn main() {
    let a = 10;
    if a < 10 {1} // Erro
    println!("{a}");
}
```

UDESC

Expressões

Em Rust boa parte das construções sintáticas são expressões:

```
fn main() {
    let a = 10;
    if a < 10 {println!(“Sim”);}
    println!("{a}");
}
```

```
fn main() {
    let a = 10;
    let b = if a < 10 {println!(“Sim”);};
    println!("{a}");
}
```

```
fn main() {
    let a = 10;
    let b = if a < 10 {println!(“Sim”);1} else {println!(“Nao”); 2};
    println!("{a}");
}
```

UDESC

while

```
fn main() {
    let mut i = 0;

    while i < 10 {
        println!("{i}");
        i += 1;
    }
}
```

```
fn main() {
    let mut i = 0;

    let r = loop {
        if i > 9 {break i;}
        println!("{i}");
        i += 1;
    };
}
```

```
fn main() {
    let mut i = 0;

    loop {
        if i > 9 {break i;}
        println!("{i}");
        i += 1;
    }; /* ; é necessário, pois uma
          expressão solta deve ter
          tipo (). */
}
```

```
fn main() {
    for i in 0..10 {
        println!("{i}");
    };
}
```

UDESC

for

```rust
fn main() {
    let mut i = 0;

    for i in 0..10 {
        println!("{i}");
    };
    println!("{i}"); // Não é o mesmo i
}
```

```rust
fn main() {
    for i in 0..10 {
        println!("{i}");
        i += 1; // Erro
    };
}
```

```rust
fn main() {
    for i in (0..10).step_by(2) {
        println!("{i}");
    };
}
```

UDESC

Declaração de funções

Em Rust uma função retorna o valor da última expressão avaliada. O *return* é um comando de fluxo de controle que encerra a função forçando o retorno do valor:

```
fn main() {
   let a = 100;

   println!("{}", maior (a, 15));
   println!("{}", fat(5));
}

fn maior (a:i32, b:i32) -> i32 {
    if a > b {a} else {b}
}

fn fat (n:u32) -> u32 {
    if n == 0 {1}
    else {n * fat(n-1)}
}
```

```
fn maior (a:i32, b:i32) -> i32 {
    if a > b
       {return a;}
    b
}

fn fat (n:u32) -> u32 {
    if n == 0
       {return 1;}
    {return n * fat(n-1);}
}
```